Project 1

ECE 485

Shashwat Choudhry

February 19, 2026

ECE 485 Project 1

Design and Simulation of a 6-to-1, 5-Bit Multiplexer

1. Introduction

A multiplexer (MUX) is a fundamental digital logic component acting as a data selector. It takes multiple input signals and forwards exactly one of them to a single output line. Multiplexers are heavily utilized in digital circuit design for data routing, parallel-to-serial conversion, and sharing a single physical communication channel or bandwidth among multiple users. By utilizing a MUX, a system can avoid using dedicated wires or radio frequencies for every single data source, instead funneling them efficiently through a shared medium.

2. Theory of Operation

The core governing principle of a multiplexer is the mathematical relationship between the number of data inputs and the number of selection lines required to control them. This relationship is defined by the formula:

$$M \leq 2^N$$

Where N represents the total number of selection lines, and M represents the maximum number of data inputs. Depending on the binary sequence applied to the selection lines, only one specific input out of the M possible inputs will be routed and made available at the output.

2.1 The 2-to-1 Multiplexer

A 2-to-1 MUX is the simplest form, requiring N = 1 selection line to choose between M = 2 inputs (I0 and I1).

Truth Table (2-to-1 MUX)

Select (S)	Output (Y)
0	I0
1	I1

2.2 The 4-to-1 Multiplexer

Scaling up, a 4-to-1 MUX requires N = 2 selection lines (S1 and S2) to choose between M = 4 inputs (I0, I1, I2, I3).

Truth Table (4-to-1 MUX)

S1	S0	Output (Y)
0	0	I0
0	1	I1
1	0	I2
1	1	I3

2.3 Constructing a 6-to-1 Multiplexer

To support 6 inputs, N=2 selection lines are insufficient (2^2 = 4). Therefore, N=3 selection lines are required (2^3 = 8 potential inputs, leaving two unused). Rather than building a massive single logic gate structure, a 6-to-1 MUX can be created using a "Structural Model." This involves cascading smaller, fundamental components; specifically, utilizing a tree architecture of five interconnected 2-to-1 multiplexers spread across three hierarchical layers to sequentially route the proper signal to the final output.

3. Problem Statement

The objective of this project is described as follows: *"A six input multiplexer is to be designed in which each input is 5 bits wide."*

Breaking this down:

- **Six Inputs:** Dictates the need for 3 selection lines and a multi-layered logical routing structure.
- **5 Bits Wide:** Instead of switching single binary bits (1s and 0s), the MUX must switch entire 5-bit data buses simultaneously. This requires defining the inputs, outputs, and internal connecting signals as STD_LOGIC_VECTOR (4 downto 0) in the hardware description language.

4. Design Workflow

The project was executed through a structured, multi-step engineering workflow:

1. **Schematic Layout:** The structural hierarchy was first diagrammed on paper. Layer 1 consisted of three parallel 2:1 MUXes. Layer 2 utilized a single 2:1 MUX, and Layer 3 used a final 2:1 MUX to consolidate the signals, carefully mapping the internal 5-bit bus lines between them.
2. **Environment Setup:** Due to macOS compatibility constraints with traditional IDEs, a terminal-based toolchain was established. The GHDL compiler was installed via Homebrew to handle the VHDL analysis and elaboration.
3. **Hardware Description:** The circuit was coded in VHDL using a structural approach. A base 5-bit 2-to-1 component was defined, and then instantiated five times in the top-level architecture to match the paper schematic.
4. **Test Bench & Simulation:** A test bench was authored to stimulate the design. Distinct hexadecimal values were assigned to the six 5-bit inputs, and a process block cycled through the 3-bit selection lines.
5. **Waveform Analysis:** The resulting .vcd file was generated via GHDL and analyzed using VCDRom, a web-based waveform viewer, to bypass local GTKWave compatibility issues and confirm the routing logic.

5. Conclusion

The structural design of the 6-to-1, 5-bit multiplexer was highly successful. Utilizing a cascading tree of 2-to-1 MUXes effectively routed multi-bit bus data without signal degradation or logic collisions. The waveform analysis clearly demonstrated that as the selection lines incremented through binary values, the exact corresponding 5-bit hexadecimal input was accurately reflected on the output bus, successfully fulfilling all parameters of the problem statement.

Images and Code

```
(base) schoudhry@dhcp178 ~ %
(base) schoudhry@dhcp178 ~ % brew install ghdl
Warning: Not upgrading ghdl, the latest version is already installed
(base) schoudhry@dhcp178 ~ % brew install --cask gtkwave
Warning: gtkwave has been deprecated because it is discontinued upstream! It was disabled on 2025-10-29.
==> Caveats
You may need to install Perl's Switch module to run gtkwave's command line
tool, e.g. using `cpan install Switch`

  https://ughe.github.io/2018/11/06/gtkwave-osx

==> Downloading https://downloads.sourceforge.net/gtkwave/gtkwave-3.3.107-osx-app/gtkwave.zip
==> Downloading from https://master.dl.sourceforge.net/project/gtkwave/gtkwave-3.3.107-osx-app/gtkwave.zip?viasf=1
######################################################################################################## 100.0%
==> Installing Cask gtkwave
==> Moving App 'gtkwave.app' to '/Applications/gtkwave.app'
==> Linking Binary 'gtkwave' to '/opt/homebrew/bin/gtkwave'
🍺  gtkwave was successfully installed!
```

Successful installation of the GTKWave application through Homebrew, confirming that GHDL is already up to date and compatible

```
zsh: killed     ghdl -a mux_design.vhd
(base) schoudhry@dhcp178 ECE485 % ghdl -a mux_design.vhd
dyld[46989]: Library not loaded: @rpath/libgcc_s.1.1.dylib
  Referenced from: <7B2DDCA4-9073-3535-A687-C71874A5D5F2> /opt/homebrew/Caskroom/ghdl/5.1.1/ghdl-llvm-5.1.1-macos15-aarch64/bin/ghdl
  Reason: tried: '/opt/homebrew/Caskroom/ghdl/5.1.1/ghdl-llvm-5.1.1-macos15-aarch64/bin/libgcc_s.1.1.dylib' (code signature in <064AC5A4-9EF3-37A9-AA5A-5057530C858E> '/opt/homebrew/Caskroom/ghdl/5.1.1/ghdl-llvm-5.1.1-macos15-aarch64/bin/libgcc_s.1.1.dylib' not vali
d for use in process: library load disallowed by system policy), '/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/aarch64-apple-darwin23.6.0/14.2.0/libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/aarch64-
apple-darwin23.6.0/14.2.0/libgcc_s.1.1.dylib' (no such file), '/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/libgcc_s.1.1.dylib' (no such file), '/Users/r
unner/work/ghdl/ghdl/gnat/lib/libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/Users/runner/work/ghdl/ghdl/gnat/lib/libgcc_s.1.1.dylib' (no such file), '/opt/homebrew/Caskroom/ghdl/6.1.1/ghdl-llvm-5.1.1-macos15-aarch64/bin/libgcc_s.1.1.dyli
b' (code signature in <064AC5A4-9EF3-37A9-AA5A-5057530C858E> '/opt/homebrew/Caskroom/ghdl/5.1.1/ghdl-llvm-5.1.1-macos15-aarch64/bin/libgcc_s.1.1.dylib' not valid for use in process: library load disallowed by system policy), '/Users/runner/work/ghdl/ghdl/gnat/lib/g
cc/aarch64-apple-darwin23.6.0/14.2.0/libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/aarch64-apple-darwin23.6.0/14.2.0/libgcc_s.1.1.dylib' (no such file), '/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/
libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/Users/runner/work/ghdl/ghdl/gnat/lib/gcc/libgcc_s.1.1.dylib' (no such file), '/Users/runner/work/ghdl/ghdl/gnat/lib/libgcc_s.1.1.dylib' (no such file), '/System/Volumes/Preboot/Cryptexes/OS/U
sers/runner/work/ghdl/ghdl/gnat/lib/libgcc_s.1.1.dylib' (no such file), '/usr/local/lib/libgcc_s.1.1.dylib' (no such file), '/usr/lib/libgcc_s.1.1.dylib' (no such file, not in dyld cache)
zsh: abort      ghdl -a mux_design.vhd
(base) schoudhry@dhcp178 ECE485 % ghdl -a mux_design.vhd
ghdl:error: exec error
```

MacOS security policy blocking the GHDL compiler from running due to an underlying library file lacking a verified Apple developer signature.

```
(base) schoudhry@dhcp178 ECE485 % ghdl -a mux_design.vhd
(base) schoudhry@dhcp178 ECE485 % ghdl -a mux_design.vhd
(base) schoudhry@dhcp178 ECE485 % ghdl -a tb_mux6_1.vhd
(base) schoudhry@dhcp178 ECE485 % ghdl -e tb_mux6_1
ld: warning: object file (/Users/schoudhry/Desktop/IIT/ECE485/e~tb_mux6_1.o) was built for newer 'macOS' version (16.0) than being linked (15.5)
ld: warning: object file (/Users/schoudhry/Desktop/IIT/ECE485/tb_mux6_1.o) was built for newer 'macOS' version (16.0) than being linked (15.5)
ld: warning: object file (/Users/schoudhry/Desktop/IIT/ECE485/mux_design.o) was built for newer 'macOS' version (16.0) than being linked (15.5)
(base) schoudhry@dhcp178 ECE485 % ghdl -r tb_mux6_1 --vcd=wave.vcd
(base) schoudhry@dhcp178 ECE485 % open -a gtkwave wave.vcd
(base) schoudhry@dhcp178 ECE485 % open -a gtkwave wave.vcd
(base) schoudhry@dhcp178 ECE485 % open -a gtkwave wave.vcd
(base) schoudhry@dhcp178 ECE485 %
```

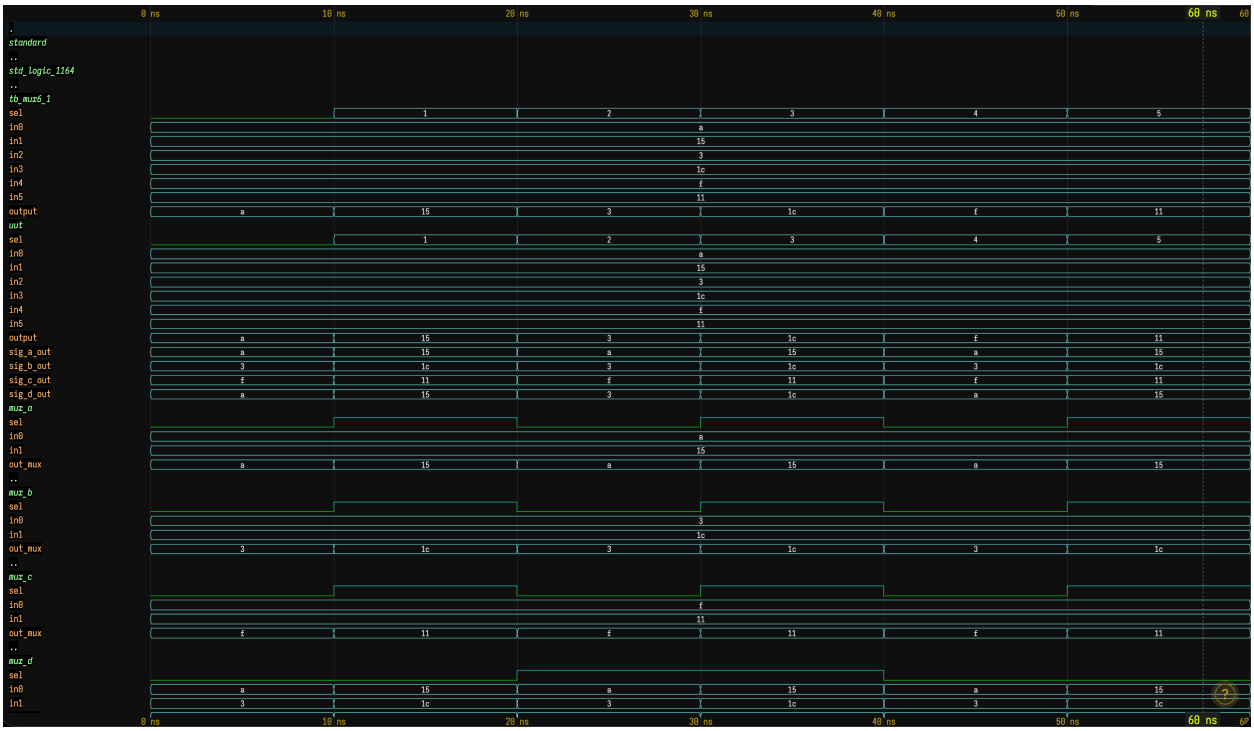
Successful compilation and simulation of your VHDL code to generate the waveform file, followed by commands attempting to open those results in GTKWave.

mux_design.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_1_5bit is
    Port ( sel : in  STD_LOGIC;
           in0 : in  STD_LOGIC_VECTOR (4 downto 0);
           in1 : in  STD_LOGIC_VECTOR (4 downto 0);
           out_mux : out STD_LOGIC_VECTOR (4 downto 0));
end mux2_1_5bit;

architecture Behavioral of mux2_1_5bit is
begin
    out_mux <= in0 when (sel = '0') else in1;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux6_1_structural is
    Port ( sel : in  STD_LOGIC_VECTOR (2 downto 0);
           in0, in1, in2, in3, in4, in5 : in  STD_LOGIC_VECTOR (4 downto 0);
           output : out STD_LOGIC_VECTOR (4 downto 0));
end mux6_1_structural;

architecture Structural of mux6_1_structural is
    component mux2_1_5bit
        Port ( sel : in  STD_LOGIC;
               in0 : in  STD_LOGIC_VECTOR (4 downto 0);
               in1 : in  STD_LOGIC_VECTOR (4 downto 0);
               out_mux : out STD_LOGIC_VECTOR (4 downto 0));
    end component;

    signal sig_A_out, sig_B_out, sig_C_out : STD_LOGIC_VECTOR (4 downto 0);
    signal sig_D_out : STD_LOGIC_VECTOR (4 downto 0);

begin
    MUX_A: mux2_1_5bit port map(sel => sel(0), in0 => in0, in1 => in1, out_mux => sig_A_out);
    MUX_B: mux2_1_5bit port map(sel => sel(0), in0 => in2, in1 => in3, out_mux => sig_B_out);
    MUX_C: mux2_1_5bit port map(sel => sel(0), in0 => in4, in1 => in5, out_mux => sig_C_out);

    MUX_D: mux2_1_5bit port map(sel => sel(1), in0 => sig_A_out, in1 => sig_B_out, out_mux => sig_D_out);

    MUX_E: mux2_1_5bit port map(sel => sel(2), in0 => sig_D_out, in1 => sig_C_out, out_mux => output);

end Structural;

```

tb_mux6_1.vhd

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_mux6_1 is
end tb_mux6_1;

architecture behavior of tb_mux6_1 is
    component mux6_1_structural
    Port ( sel : in  STD_LOGIC_VECTOR (2 downto 0);
           in0, in1, in2, in3, in4, in5 : in  STD_LOGIC_VECTOR (4 downto 0);
           output : out STD_LOGIC_VECTOR (4 downto 0));
    end component;

    signal sel : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal in0 : STD_LOGIC_VECTOR(4 downto 0) := "01010";
    signal in1 : STD_LOGIC_VECTOR(4 downto 0) := "10101";
    signal in2 : STD_LOGIC_VECTOR(4 downto 0) := "00011";
    signal in3 : STD_LOGIC_VECTOR(4 downto 0) := "11100";
    signal in4 : STD_LOGIC_VECTOR(4 downto 0) := "01111";
    signal in5 : STD_LOGIC_VECTOR(4 downto 0) := "10001";
    signal output : STD_LOGIC_VECTOR(4 downto 0);

begin
    uut: mux6_1_structural port map (
        sel => sel, in0 => in0, in1 => in1, in2 => in2,
        in3 => in3, in4 => in4, in5 => in5, output => output
    );

    stim_proc: process
    begin
        sel <= "000"; wait for 10 ns;
        sel <= "001"; wait for 10 ns;
        sel <= "010"; wait for 10 ns;
        sel <= "011"; wait for 10 ns;
        sel <= "100"; wait for 10 ns;
        sel <= "101"; wait for 10 ns;
        wait;
    end process;
end behavior;

```



Displays the final waveform simulation of the 6-to-1 multiplexer, confirming that the 5-bit output successfully routes each corresponding input as the select signal changes.

References

[1] GeeksforGeeks, "Common Bus System using Multiplexers." [Online]. Available: https://www.geeksforgeeks.org/digital-logic/common-bus-system-using-multiplexers/.

[2] HowStuffWorks, "How Microprocessors Work." [Online]. Available: https://computer.howstuffworks.com/microprocessor.htm.

[3] Analog Devices, "Interface a Multiplexed-Bus Real-Time Clock to a µP with Separate Address and Data Buses." [Online]. Available: https://www.analog.com/en/resources/design-notes/interface-a-multiplexedbus-realtime-clock-to-a-181p-with-separate-address-and-data-buses.html.

[4] J. Valvano, "Digital Logic," *EE445L eBook*, University of Texas at Austin. [Online]. Available: https://users.ece.utexas.edu/~valvano/EE445L/ebook/DigitalLogic.html.

[5] Xilinx, "Multiplexers," in *Virtex-II Platform FPGA Handbook*, Q1-2002. [Online]. Available: http://ebook.pldworld.com/_semiconductors/Xilinx/DataSource%20CD-ROM/Rev.6%20(Q1-2002)/userguides/V2_handbook/ug002_ch2_multiplexers.pdf.

[6] AMD, "Designing Large Multiplexers," *7 Series FPGAs Configurable Logic Block User Guide*. [Online]. Available: https://docs.amd.com/r/en-US/ug474_7Series_CLB/Designing-Large-Multiplexers.

[7] Texas Instruments, "Analog Multiplexers and Demultiplexers," Application Note SCDA047. [Online]. Available: https://www.ti.com/lit/an/scda047/scda047.pdf.